



Progettazione di un programma

Massimo Esposito



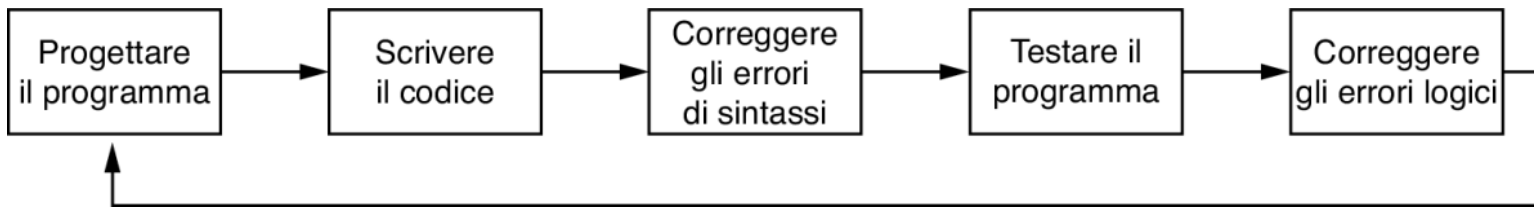
Il ciclo di sviluppo di un programma



Sommario

1. Il ciclo di sviluppo di un programma
 2. Fasi della progettazione di un programma
 3. Strumenti per progettare: pseudocodice e diagrammi di flusso
- 

Fasi del ciclo di sviluppo



Il **ciclo di sviluppo** di un programma è composto da cinque fasi distinte.

- Ogni fase ha un ruolo critico e contribuisce alla **qualità complessiva** del programma finale.
- Questo processo garantisce che il software sia **funzionale, affidabile** ed **efficiente**.

Progettazione

- La fase di progettazione rappresenta la **base fondamentale** su cui si costruisce il programma.
- Una progettazione accurata consente di risparmiare tempo in futuro, **prevenendo modifiche complesse**.
- I professionisti concordano sul fatto che un buon progetto riduce il **rischio di errori** e migliora la **manutenibilità**.

Scrittura del codice

- Scrivere il codice avviene solo dopo una **solida progettazione**.
- Il codice è scritto in linguaggi ad alto livello, come **Python**, seguendo **regole sintattiche precise**.
- Le **parole chiave**, gli **operatori** e la **punteggiatura** devono essere usati con attenzione per evitare errori.

Correzione degli errori di sintassi

- Gli **errori di sintassi** si verificano frequentemente alla prima stesura del codice.
- I compilatori o interpreti **segnalano** questi errori per permettere la loro **correzione**.
- La **rimozione di tali errori** è essenziale per poter compilare o eseguire correttamente il programma.

Testing del programma

Dopo che il codice è eseguibile, si procede con il **test** per individuare errori logici.

Un **errore logico** produce risultati scorretti pur non bloccando l'esecuzione del programma.

- Gli errori matematici sono una causa molto comune di errori logici.

Debugging del programma

- Il **debug** consiste nell'individuare e correggere questi problemi, anche **ristrutturando** il programma se necessario.
- Il programmatore può scoprire che è meglio modificare **la struttura originaria del programma**.
- In questo caso, il ciclo di sviluppo del programma **ricomincia e continua fino a che non si trovano più errori**.



Fasi della progettazione di un programma

La progettazione in dettaglio

La fase di progettazione di un programma può essere riassunta in questi **due passaggi**:

- capire che lavoro deve svolgere il programma;
- determinare i passaggi da seguire per svolgere il lavoro.

Analisi dei requisiti

- Comprendere il **compito del programma** è il primo passo per una progettazione efficace.
- I programmatori devono chiarire esattamente cosa il software deve fare, analizzando gli obiettivi richiesti.
- Solo attraverso un'analisi accurata si possono stabilire **funzionalità chiare** e **ben definite**.

Il ruolo del cliente

- Il **cliente** è colui che commissiona il programma e ne definisce le necessità.
- Può trattarsi di un committente esterno o di un responsabile interno all'azienda.
- Stabilire una comunicazione chiara con il cliente è essenziale per garantire **coerenza e soddisfazione**.

Colloqui e raccolta informazioni

- I programmatori incontrano il cliente per raccogliere **informazioni dettagliate** sui requisiti.
- Le domande servono a chiarire ogni aspetto del compito che il software dovrà svolgere.
- Spesso sono necessari **più incontri** per definire completamente i requisiti.

Definizione dei requisiti software

- Le informazioni raccolte vengono organizzate in un elenco di **requisiti del software**.
- Ogni **requisito** rappresenta **una singola funzionalità** che il programma deve realizzare.
- Quando il cliente approva l'elenco, si può passare alla fase di **progettazione dell'algoritmo**.

Scomposizione del problema

- Una volta compreso il compito, è necessario **scomporlo in passaggi semplici e ordinati**.
- Questo aiuta a rendere il programma più chiaro e facilmente traducibile in codice.
- Una buona scomposizione facilita la **comprensione e la manutenzione del software**.

Creare un algoritmo

- Un **algoritmo** è una serie logica e sequenziale di operazioni necessarie per risolvere un problema.
- Deve essere preciso, completo e seguire un ordine specifico.
- Un buon algoritmo rappresenta la **struttura logica** del programma futuro.

Esempio: calcolo della retribuzione

Su supponga di dover scrivere un programma che **calcoli e visualizzi l'emolumento lordo di un dipendente a ore.**

I passaggi da dover seguire sono i seguenti:

1. ottenere il numero di ore lavorate;
2. ottenere la retribuzione oraria;
3. moltiplicare il numero di ore lavorate per la retribuzione oraria;
4. visualizzare il risultato del calcolo eseguito al passaggio 3.

Questo esempio mostra come tradurre **un problema reale in una sequenza operativa.**

Importanza della sequenza

- **La sequenza è fondamentale:** ogni passaggio deve seguire il precedente in modo logico.
- Alterare l'ordine **comprometterebbe** la correttezza del programma.
- **Mantenere un ordine preciso** è una regola chiave nello sviluppo di algoritmi.



Strumenti per progettare: pseudocodice e
diagrammi di flusso

Cos'è lo pseudocodice

- Lo **pseudocodice** è un linguaggio informale utilizzato per descrivere le operazioni di un programma.
- Non segue regole sintattiche rigide, il che lo rende flessibile e comprensibile.
- Viene utilizzato per **pianificare il codice** prima di scriverlo in un linguaggio formale come Python.

Vantaggi dello pseudocodice

- Uno dei principali vantaggi dello pseudocodice è che **evita errori di sintassi** durante la fase iniziale.
- Permette ai programmatori di concentrarsi sulla **logica del programma** senza preoccuparsi della forma.
- È uno strumento utile anche per **comunicare l'idea del programma** ad altri sviluppatori.

Esempio di pseudocodice

Inserire le ore lavorate

Inserire la retribuzione oraria

Calcolare la retribuzione lorda come numero di ore lavorate moltiplicate per la retribuzione oraria

Visualizzare la retribuzione lorda

Nell'esempio di pseudocodice per il **programma di calcolo della retribuzione** mostrato in Figura:

- ogni frase rappresenta un'istruzione che può essere convertita facilmente in codice Python.

Cos'è un diagramma di flusso

- Un **diagramma di flusso** rappresenta graficamente i passaggi di un programma.
- Utilizza **forme geometriche standard** per visualizzare operazioni, input/output e flusso logico.
- Aiuta a comprendere visivamente la **struttura** e la **sequenza** delle operazioni.

Simboli nei diagrammi di flusso

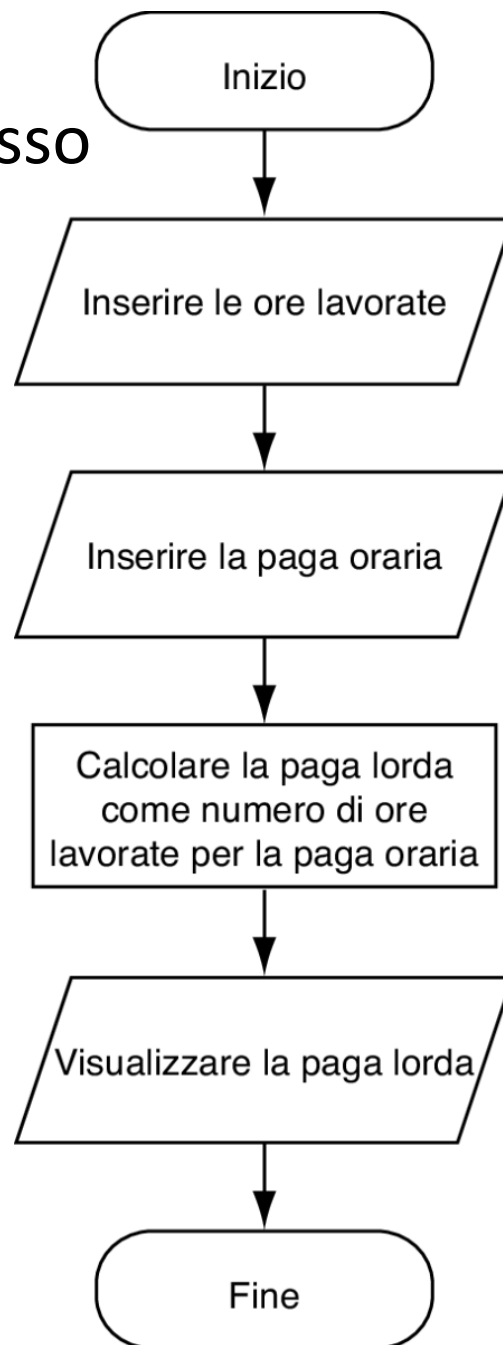
Si possono individuare tre tipi principali di blocchi:

- **Blocchi terminali (ovali)**: Sono posti all'inizio e alla fine del diagramma.
- **Blocchi di input/output (parallelogrammi)**: Sono utilizzati per rappresentare le operazioni di lettura dei dati (input) e le operazioni di visualizzazione dei risultati (output)
- **Blocchi di elaborazione (rettangoli)**: Mostrano i passaggi in cui il programma esegue operazioni o calcoli.

Tutti questi blocchi sono collegati da **frecce**, che indicano la direzione del flusso del programma.

- Seguendo le frecce dal blocco **Inizio** al blocco **Fine**, si percorre l'intero processo logico dell'algoritmo.

Esempio di diagramma di flusso



Nell'esempio di diagramma di flusso per il **programma di calcolo della retribuzione** mostrato in Figura:

- Ogni operazione va eseguita nell'ordine in cui è rappresentata nel diagramma.
- Il diagramma facilita l'identificazione di **passaggi ridondanti o errati** durante la progettazione.

Bibliografia

Libro di testo

- Tony G. (2022). Introduzione a Python - 5/Ed (Capitolo 2, pp. 48-52). Pearson Italia spa. (Disponibile nella sezione “Biblioteca”)

Crediti

- Tutte le immagini utilizzate in questa presentazione sono tratte dal libro di testo qui indicato in bibliografia.